

Student Notes

Batch: IITP-AIMLTN-2605 · **Program:** Applied Artificial Intelligence and Machine Learning Essentials (Tamil) - i-HUB IIT Patna

This guide compiles the key concepts you need for your upcoming evaluation. Work through each section, focus on the worked examples and definitions, and use the key terms to self-test. Sections are ordered from foundations to more advanced material.

1. The AI / ML Landscape

Artificial Intelligence (AI) is a system or machine designed to mimic human intelligence to perform tasks faster or better. **Machine Learning (ML)** is a subset of AI that learns patterns from data and makes predictions based on past data — models are trained on datasets and then used to predict on unseen data. **Generative AI (Gen AI)** is a further subset of ML that generates new, unseen content (text, images, code); ChatGPT and Gemini are examples, built as Large Language Models (LLMs) trained on massive datasets. The hierarchy nests: **AI** $\supset$ **ML** $\supset$ **Gen AI** — not every ML model is generative, but every generative model is an ML model.

History of AI

- **1950s** — Alan Turing introduces the **Turing Test**: a machine is deemed intelligent if its behavior is indistinguishable from a human's.
- **1956** — The **Dartmouth Conference** coins the term “Artificial Intelligence.”
- **1960s–1980s** — Rule-based systems such as **ELIZA** simulate conversation using predefined if-else rules.
- **1980s — AI Winter** — Research funding declines due to the limitations of rule-based AI and unrealistic expectations.
- **1990s–2000s** — The internet boom (Google, Amazon) brings an explosion of data and computational power, renewing interest in ML algorithms.
- **2010s–present** — Deep learning and neural networks, powered by GPUs and massive datasets, lead to the 2020s rise of transformer architectures and Gen AI models like ChatGPT and Gemini.

Computer & Programming Fundamentals

- **Hardware** — physical components: keyboard, mouse, CPU, RAM, storage, monitor.
- **Software** — programs and operating systems that instruct hardware; analogous to the “brain.”
- **Program** — a set of instructions for a computer. **Programming** — the act of writing those instructions. **Programming languages** (Python, C++, Java) are formal languages for this.

- **Interpreter vs Compiler** — an interpreter (Python) executes code line-by-line, useful for step-by-step debugging; a compiler (C++) translates the entire program before execution.
- **IDE** (VS Code) — for writing/running/debugging code. **CLI** — command-line interaction with the OS.
- **File types:** .py (Python files), .ipynb (Jupyter notebooks), .csv, .txt.
- **PIP** — Python's package installer, used to install libraries like NumPy and pandas. **Virtual environments** isolate project dependencies (detailed treatment in Section 2).
- **API keys** — secret authentication tokens for external services; never share them publicly, as this risks unauthorized access.
- *(Briefly mentioned, not detailed here: the course roadmap spans Python basics → data analysis → ML algorithms → deep learning → LLMs → Gen AI applications, building toward an interview-ready portfolio.)*

Key terms: AI, ML, Gen AI, Turing Test, Dartmouth Conference, ELIZA, AI Winter, interpreter, compiler, IDE, CLI, API key.

2. Developer Toolkit: Terminal & Git

Terminal (PowerShell) Basics

- **Navigation:** `pwd` (print working directory), `ls/dir` (list contents), `cd folder_name` (change directory), `cd ..` (up one level).
- **File/folder operations:** `mkdir folder_name` (create folder), `rm/rmdir directory_name` (remove), `rm -rf folder_name` (recursively delete a directory and everything inside — use cautiously), `ni filename.txt/ni filename.py` (create a file), `cp source destination` (copy), `mv oldname newname` (rename/move).
- **Other:** `cls/clear` (clear the screen).
- *(Mentioned: Mac/Linux equivalents of these commands exist and can be looked up separately; detailed installation and advanced Git usage — e.g. rebasing, stashing, conflict resolution — are beyond this module's coverage.)*

Python Virtual Environments

- **Purpose:** isolate per-project dependencies so different projects can use different library versions without conflict.
- **Lifecycle:**
 1. Create: `python -m venv env_name`
 2. Activate: Windows `.\env_name\Scripts\activate`; Mac/Linux `source env_name/bin/activate`
 3. Install libraries inside the active environment: `pip install package_name`
 4. Deactivate: `deactivate`

- **Benefits:** avoids dependency conflicts, manages different Python versions per project, and `pip freeze` shares the environment setup for reproducibility across a team.

Git & Version Control

- **Git** tracks changes to files, enabling version control. **GitHub/GitLab** host remote repositories in the cloud.
- **Core commands:**
 - `git init` — initialize a repository locally
 - `git add .` — stage all changes
 - `git commit -m "message"` — commit staged changes
 - `git push origin main` — push commits to the remote main branch
 - `git clone <url>` — clone a remote repository locally
 - `git status` — check working directory / staging area state
- **Branching:**
 - `git branch` — list branches
 - `git checkout -b branch_name` — create and switch to a new branch
 - `git checkout main` — switch back to main
 - `git branch -d branch_name` — delete a branch (typically after merging)
- **Collaboration:** developers branch off to work separately, then open **pull requests** to merge into main; collaborators can be added to a repository for team access.
- **Inspecting an older commit:** `git checkout <commit_hash>` checks out that snapshot in a detached HEAD state; it does not move the branch pointer or permanently roll back history.

Python Basics: Variables, Data Types, Conditionals

- **Variables** are named containers; assignment uses `=` (not `==`, which is comparison). Names must start with a letter/underscore, contain no spaces/special characters (except `_`), and are case-sensitive (`venki` $\neq$ `Venki`).
- **Data types:** `int` (whole numbers), `float` (decimals), `str` (text). Use `type(variable)` to check a value's type. Variables can be reassigned to different types dynamically.
- **Conditional statements:**

```
if cgpa > 9:
    grade = 'A'
elif cgpa > 7:
    grade = 'B'
else:
    grade = 'F'
```

Only the first satisfied condition executes; the rest are skipped. Indentation (commonly 4 spaces) defines code blocks — indentation errors cause syntax errors. Keywords (`if`, `for`, `class`, ...) cannot be used as variable names, and variables must be defined before use.

Key terms: pwd, cd, mkdir, rm -rf, venv, pip, pip freeze, git init/add/commit/push/clone/status, branch, pull request, if/elif/else, indentation.

3. Python Data Structures: Lists & Tuples

Typecasting & Recap

- Explicit type conversion: `int()`, `float()`, `str()`. Converting a **float to an int truncates the decimal part toward zero** (e.g. `int(9.8) → 9`); for positive numbers this coincides with flooring (rounding down), but they are technically different operations (they diverge for negative numbers). String-to-int only works if the string represents a whole number; otherwise convert to float first. Strings are **immutable** — reassigning a string variable creates a new object at a new memory address.

Lists

- **Ordered, mutable**, can hold heterogeneous types, created with `[]`, length via `len()`. Lists support nesting (a list inside a list).
- **Indexing:** starts at 0 (positive) or `-1` for the last element (negative). Out-of-range indexing raises `IndexError`.
- **Slicing:** `start:stop:step` — `stop` is exclusive, `step` can be negative to reverse; `step` cannot be zero. Out-of-range *slicing* does not error (unlike out-of-range *indexing*).
- **Methods:** `append(x)` (add one element at the end), `insert(index, x)` (add at a position), `extend(iterable)` (add multiple elements from an iterable — note: extending with a *string* adds each character separately, since strings are iterable), `pop()` (remove & return last element, or a specified index), `remove(x)` (remove first occurrence of a value), `clear()` (empty the list), `count(x)`, `index(x)`.
- **sort() vs sorted():** `sort()` sorts in place and returns `None`; `sorted()` returns a new sorted list, leaving the original untouched.
- **Mutability & aliasing:** `L2 = L1` copies the *reference* — both names point to the same list, so modifying one affects the other. Use `L1.copy()` to create a genuinely independent list.

Tuples

- **Ordered, immutable**, created with `()`. Once created, contents cannot be modified.
- Single-element tuples need a trailing comma: `(10,)`.
- Only support non-modifying methods: `count()`, `index()` — no `append()`/`extend()`.
- To “modify” a tuple: convert to a list, change it, convert back to a tuple.
- Ideal for fixed data (coordinates, constants). A tuple *can* contain a mutable object like a list — that inner list can still be modified even though the tuple itself cannot be changed.

Key terms: `IndexError`, slicing, `append/extend/insert/pop/remove`, `sort` vs `sorted`, aliasing vs `copy`, tuple immutability.

4. Python Data Structures: Sets, Frozensets & Mutability

Memory & Mutability Recap

Variables store **references** to values, not the values themselves. Lists are mutable (contents change at the same memory address); tuples, strings, and numbers are immutable (a “change” creates a new object at a new address).

Sets

- An **unordered collection of unique elements**; mutable. Adding a duplicate is silently ignored (no error). Sets do not support indexing or slicing.
- **Methods:** `add(x)`, `remove(x)` (errors if `x` absent), `discard(x)` (no error if `x` absent), `pop()` (removes & returns an arbitrary element, since sets are unordered), `clear()`, `update(iterable)`.
- **Set operations:** union (combine both sets), intersection (elements common to both), subset (all elements of one set are in another), superset (contains all elements of another). Methods: `update()`, `intersection_update()`, `issubset()`.
- Sets can contain tuples (hashable) but **cannot** contain lists or other sets (mutable → unhashable).

Hashability

- **Hashable objects** have a hash value that never changes over their lifetime — this requires immutability. Integers, floats, strings, and tuples-of-hashable-elements are hashable.
- **Mutable objects (lists, sets)** cannot be hashed, so they cannot be set elements or dictionary keys.

Frozen Sets

- Like a set, but **immutable** — no `add()`/`remove()`. Because they’re immutable, frozensets are hashable, so a frozenset can itself be an element of another set (a regular mutable set cannot). Convertible to/from tuples and lists.

Key terms: set, discard vs remove, union/intersection/subset/superset, hashable, frozenset.

5. Dictionaries, Loops & Control Flow

Dictionaries

- A collection of **key-value pairs**; created with `{}` or `dict()`. Keys must be **unique** and **immutable** (string, int, float, tuple, frozenset); values may repeat and be any type.
- Since Python 3.7, dictionaries maintain **insertion order**.
- **Operations:** access via `d['key']`, add/update via assignment `d['key'] = value`, delete via `del d['key']` or `d.pop('key')`; `.keys()`, `.values()`, `.items()` return keys/values/key-value pairs.

- Assigning a **duplicate key overwrites** the existing value (no error, no second copy).

Loops

- **For loop:** iterates over any iterable (list, tuple, set, dict, string). Iterating a dict directly loops over its **keys** by default — use `.values()` or `.items()` for the other views. Supports an **else clause**: runs only if the loop completes *without* hitting `break`.
- **While loop:** runs as long as its condition is true; must update the controlling variable to avoid infinite loops.
- **Nested loops:** the inner loop completes all its iterations for every single iteration of the outer loop.

Control-flow statements

- **break** — exits the loop immediately (and skips any `else` clause).
- **continue** — skips the rest of the current iteration, moves to the next.
- **pass** — a no-op placeholder for syntactically-required but empty code blocks.

`range()`

`range(start, stop, step)` — `start` defaults to 0, `stop` is required (exclusive), `step` defaults to 1 (can be negative to count down). Wrap in `list()/tuple()` to materialize the sequence.

Key terms: key-value pair, insertion order, for/while, for-else, break/continue/pass, `range()`.

6. Functions

- **Definition:** `def function_name(parameters):` — the body must be indented. A function only runs when called: `function_name()`. Names follow variable-naming rules and are case-sensitive.
- **Parameters vs arguments:** parameters are the placeholders in the definition; arguments are the actual values passed at the call site.
- **Default parameter values** make an argument optional: `def greet(country="India"):`
- **Keyword arguments** can be passed by name in any order; but when mixing styles, **all positional arguments must come before any keyword arguments** in the call.
- **return vs print:** `return` sends a value back to the caller and ends execution; a function with no `return` implicitly returns `None`. `print` only displays output to the console — it does not hand back a usable value.
- **Arbitrary arguments:**
 - `*args` — collects any number of extra **positional** arguments into a **tuple**.

```
def calculate_bill(*prices):
    for price in prices:
        print(price)
```

- ****kwargs** — collects any number of extra **keyword** arguments into a **dictionary**.

```
def user_info(**details):
    for key, value in details.items():
        print(f"{key}: {value}")
```

- **Positional-only (/) and keyword-only (*) markers** in a signature restrict how certain parameters may be passed; mixing these must follow Python's required ordering rules.
- **Variable scope:** a variable declared inside a function is **local**; one declared outside is **global**. Use the `global` keyword inside a function to modify a module-level variable:

```
x = 300
def func():
    global x
    x = 100
func()
print(x) # 100
```

Key terms: parameter vs argument, default value, keyword argument, `*args`, `**kwargs`, positional-only/keyword-only, `global` keyword.

7. Variable Scope (LEGB), File Handling & JSON

Scope and the LEGB Rule

- **Global** variable — defined outside any function, accessible everywhere (read freely; needs `global` to modify from inside a function).
- **Local** variable — defined inside a function, accessible only there.
- **Non-local** variable — used in nested functions to access/modify an enclosing function's variable; requires the `nonlocal` keyword to modify (not just read).
- **LEGB** — Python resolves a name in this order: **Local** → **Enclosing** → **Global** → **Built-in**.

File Handling

- **Modes:** `r` (read, default), `w` (write — creates new or **overwrites** existing content), `a` (append — adds without deleting existing data), `x` (exclusive create — fails if the file exists).
- **Reading:** `read()` (entire content), `readline()` (one line), `readlines()` (list of all lines).
- **Context manager (with)** automatically opens and closes a file, removing the need to call `close()` explicitly — this works for reading, writing, and appending alike.

JSON

- A key-value text format like Python dicts, but: JSON uses lowercase `null/true/false` (Python: `None/True/False`) and requires double-quoted strings.
- **json module:** `json.load(file)` reads JSON from a file into Python objects; `json.loads(string)` parses a JSON string; `json.dump(obj, file)` writes a Python

object as JSON to a file; `json.dumps(obj)` converts a Python object to a JSON string. Useful for nested structures like lists of dictionaries (transactional/employee records, etc.).

Key terms: LEGB, nonlocal, file modes, with statement, `json.load/loads/dump/dumps`.

8. NumPy Fundamentals

- **NumPy** is optimized for fast numerical computation on large datasets (transaction records, etc.), using low-level C code — often 50×+ faster than equivalent Python-list loops.
- **ndarray** — NumPy's core data structure; can be 1D, 2D, or 3D. Requires **homogeneous data types**; mixing types forces conversion (often to strings), which hurts numerical efficiency.
- **Vectorized operations** apply to an entire array without an explicit loop: `np_array + 10`, `np_array ** 2`.
- **Reshaping:** change an array's shape while preserving total element count (e.g. $3 \times 2 \rightarrow 2 \times 3$, 1×6 , or 6×1). `reshape(-1, 1)` lets NumPy auto-calculate one dimension while the other is fixed — handy when the exact shape is unknown (common in ML preprocessing); e.g. a 12-element array reshaped with `reshape(-1, 3)` becomes shape `(4, 3)` since $12 / 3 = 4$.

Key terms: ndarray, vectorized operation, homogeneous dtype, `reshape(-1, n)`.

9. Pandas Fundamentals: Series & DataFrames

- **Pandas Series** — like a NumPy array but with **indexed labels** (e.g. city names), more human-readable; supports vectorized operations.
- **Pandas DataFrame** — a 2D labeled structure (rows + columns), like a spreadsheet or SQL table; each column can hold a different data type.
- **Creating DataFrames:** from dictionaries (keys $\rightarrow$ columns), from CSV files, etc.
- **Viewing data:** `.head()` / `.tail()` (first/last rows), `.info()` (columns, dtypes, non-null counts — key for spotting missing data).
- **Column selection:** a single column with single brackets `df['col']` returns a **Series**; one or more columns with double brackets `df[['col1', 'col2']]` (or even a single column in double brackets `df[['col']]`) returns a **DataFrame**.
- **Missing values** appear as NaN ("not a number"), analogous to null in databases/Excel.

Key terms: Series vs DataFrame, single vs double bracket selection, NaN.

10. Data Cleaning: Indexing, Missing Values, Duplicates & Types

`.iloc[]` vs `.loc[]`

- `.iloc[]` — integer position-based; **end index exclusive** (like Python ranges).
- `.loc[]` — label-based; **both start and end are inclusive** — this can produce a *different row count* than `.iloc[]` even when the index looks like plain integers.
- **Conditional filtering** with `.loc[]` uses boolean masks, combined with `&` (and) / `|` (or) — each condition wrapped in parentheses, e.g. `df.loc[(df['age'] < 18) & (df['sex'] == 'male')]`. (Python's plain and/or keywords do not work element-wise on a pandas boolean Series; `=` inside a condition is an assignment, not a comparison, and is invalid there.)

Missing Values

- **Types:** numerical or categorical columns can each have missing entries.
- **Imputation:** numerical → mean (if no major outliers) or median (robust to outliers/skew); categorical → mode (most frequent value), or mark as 'unknown' instead of guessing. Constants like `-1/0` can also be used where contextually meaningful.
- **Group-wise imputation** (e.g. via `groupby(...).transform('mean')/'median'`) is often more accurate than a single overall statistic, when the value varies meaningfully across a subgroup (e.g. imputing age by passenger class).
- **Dropping:** drop rows with missing values in non-critical columns; drop entire columns once the missing proportion is very large (commonly-cited rule of thumb: somewhere north of ~30%, though the exact cutoff is a judgment call, not a fixed law — the larger the missing share, the more imputation would mean fabricating the column).
- **Using other features** to impute (e.g. estimate age from passenger class) uses a relationship in the data rather than a blind global statistic.
- **Data validation** — check for impossible/mismatched values (e.g. `age = 280`, negative values where impossible); correct or remove them.

Duplicates

- **Exact duplicates** — every column identical. **Partial duplicates** — same key (e.g. order ID) but different other fields (timestamp/status). **Near duplicates** — small variations (case differences like “Apple” vs “apple”, or typos).
- `.duplicated()` identifies duplicate rows; `drop_duplicates()` removes them. Use `keep='first'/'keep='last'` (after sorting, e.g. by timestamp) to control which copy survives — e.g. sort ascending by timestamp and use `keep='last'` to retain each key's most recent status.
- Normalize text (`.str.lower()`, `.str.strip()`) before comparing, to catch near-duplicates caused by case/whitespace.

Data Types

- Numeric values stored as strings can't be used for calculations; dates stored as strings need conversion.
- `pd.to_datetime()` converts strings to datetime — for columns that genuinely mix date layouts (e.g. 2026-01-05 and 25-01-2026), pass `format="mixed"` explicitly, since recent pandas versions can otherwise raise an error on unspecified mixed formats.
- `astype(float)` converts to float. `pd.to_numeric(errors='coerce')` converts to numeric, turning unconvertible entries into NaN (which `fillna()` can then patch).
- **Cleaning categorical variants:** `.str.lower()` / `.str.strip()` to normalize, then `.replace()` / `.map()` to unify typos/abbreviations to one standard label (e.g. "m", "M", "male" → "male"). *(Converting a cleaned categorical column into a numeric encoding for modeling — e.g. one-hot or label encoding — is a separate step covered later in the course, not here.)*

Key terms: `iloc` vs `loc`, boolean mask, mean/median/mode imputation, group-wise imputation, exact/partial/near duplicate, `drop_duplicates keep=`, `to_datetime`, `to_numeric(errors='coerce')`.

11. Advanced Pandas: GroupBy, Aggregation, Transform & Pivot Tables

Sorting & Filtering Recap

- `sort_values()` sorts by one or more columns, each with its own ascending/descending order via a matching list, e.g. `df.sort_values(by=['city', 'sales'], ascending=[True, False])`.
- `.isin([...])` filters rows matching any value in a list.

GroupBy

- `df.groupby('col')` groups rows by a column's values (like SQL's `GROUP BY`).
- `.groups` exposes a mapping from each group key to its row indexes; `.get_group('key')` extracts one group's rows as a standalone DataFrame.
- **Aggregations:** `sum()`, `mean()`, `count()` (non-missing values only), `size()` (every row in the group, regardless of missing values in any particular column) — these two can diverge when a column has NaNs.
- **Named aggregations** via `.agg()` let you apply different functions to different columns and rename the outputs in one call:

```
df.groupby('city').agg(  
    sales_sum=('sales', 'sum'),  
    profit_mean=('profit', 'mean'),  
    order_count=('orderID', 'count'),  
)
```

- **Grouping by multiple columns** (e.g. `groupby(['city', 'category'])`) produces **tuple** group keys, e.g. `('Pune', 'Electronics')`.

transform() vs .agg()

- `.agg()` **reduces** — one output row per group.
- `.transform()` **broadcasts** — keeps the original row count, repeating each group's aggregate value across every row in that group. This is exactly why `transform()` (not `agg()`) is the tool for group-wise imputation: `df['age'].fillna(df.groupby('pclass')['age'].transform('mean'))`.

Pivot Tables

- A **matrix / cross-tabulated view** of grouped data (like an Excel pivot table): rows (index), columns (columns), and aggregated values.

```
pd.pivot_table(df, index='region', columns='category', values='revenue',
aggfunc='sum', fill_value=0)
```

- `fill_value` fills empty region-category combinations (avoiding NaN); the **default aggfunc is 'mean'**, so you must specify 'sum' explicitly for totals.
- `values` can take a **list** of columns to aggregate multiple measures (e.g. `['sales', 'profit']`) at once.
- `margins=True` adds an “All” grand-total row/column. The margin always uses the same `aggfunc` as the rest of the call — since the default `aggfunc` is 'mean' (not 'sum'), an unspecified call gives a **mean** margin; pass `aggfunc='sum'` explicitly if you want the “All” total to be a sum instead.
- Multi-level pivot-table columns can be **flattened** (e.g. via list comprehension + string joins) into single string labels for simpler downstream use.

Key terms: `groupby`, `get_group`, `count` vs `size`, named aggregation, `transform` vs `agg`, `pivot_table`, `margins`, `fill_value`.

12. SQL Fundamentals

Why SQL

SQL efficiently queries/filters large datasets (millions of records) that can't be loaded directly into memory; pandas is then used for in-memory analysis on the resulting, more manageable subset. SQLite (usable inside Google Colab) is a lightweight way to practice; pandas' `read_sql_query` runs SQL against such a database and returns a `DataFrame`.

Core clauses

- **SELECT** — choose columns. **FROM** — the table. **WHERE** — filters **rows** (evaluated before any grouping).
- **GROUP BY** — aggregates rows into groups (e.g. total spend per customer).

- **HAVING** — filters **groups**, evaluated *after* aggregation (e.g. keep only customers whose total spend exceeds ₹20,000). WHERE and HAVING can be combined in one query: WHERE narrows raw rows first, then HAVING narrows the aggregated result.
- **Logical execution order:** FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

Database design

- **Normalization** — splitting data into separate tables (e.g. customers, orders) to avoid redundancy.
- **Primary key** — a unique identifier for each row (e.g. customer ID). **Foreign key** — a column in one table referencing another table's primary key (e.g. customer_id in orders pointing to customers).

SQL ↔ Pandas equivalence

SQL	Pandas
SELECT col1, col2 FROM t	df[['col1', 'col2']]
WHERE sales > 10000	df[df['sales'] > 10000]
ORDER BY sales DESC LIMIT 3	df.sort_values('sales', ascending=False).head(3)
WHERE status='completed' AND city='Mumbai'	df[(df['status']=='completed') & (df['city']=='Mumbai')]

Key terms: WHERE vs HAVING, execution order, normalization, primary/foreign key.

13. SQL Joins, Subqueries & CTEs

Joins

Combine rows from two or more tables on a related key — essential because normalized data (customers/orders/products) is spread across multiple tables. - **Inner join** — only rows with matching keys in both tables. - **Left join** — all rows from the left table, with NULL filled in where the right table has no match. - **Right join** — mirror image of a left join (keeps all rows from the right table). - **Full join** — all rows from both tables, NULL filling in wherever a match is missing. - Joined results are **virtual** (not physically stored) and automatically reflect any updates to the underlying tables.

Subqueries

A query nested inside another, executed first, with its result feeding the outer query — e.g. finding products priced above the average product price:

```
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

This avoids hardcoding a value (like “the average”) that would go stale as data changes.

Common Table Expressions (CTEs)

A **WITH** clause defines a **temporary, named result set**, scoped to a single query, to make complex multi-step queries more readable and reusable — instead of one deeply nested subquery. CTEs can be **chained**: one CTE built on top of another.

```
WITH customer_spending AS (  
    SELECT customer_id, SUM(price * quantity) AS total_spent  
    FROM orders JOIN products ON orders.product_id = products.product_id  
    GROUP BY customer_id  
)  
average_spending AS (  
    SELECT AVG(total_spent) AS avg_spent FROM customer_spending  
)  
SELECT c.customer_name, cs.total_spent  
FROM customer_spending cs  
JOIN customers c ON cs.customer_id = c.customer_id  
WHERE cs.total_spent > (SELECT avg_spent FROM average_spending);
```

Key terms: inner/left/right/full join, subquery, CTE (WITH clause), chained CTEs.

14. SQL Window Functions & Exploratory Data Analysis (EDA)

Window Functions

- Perform a calculation across a set of related rows **without collapsing** the original table — unlike **GROUP BY**, which aggregates and discards row-level detail. Every original row survives, each annotated with its group's aggregate.
- **OVER (PARTITION BY department_id)** groups rows by department while preserving each individual row.
- **Ranking functions:**
 - **RANK()** — tied values share a rank; the next rank **skips** by the number of ties (e.g. 1, 1, 3).
 - **DENSE_RANK()** — tied values share a rank; the next rank does **not** skip (e.g. 1, 1, 2).
 - **ROW_NUMBER()** — always assigns unique, strictly increasing numbers, even to tied values.
- Window functions can be layered **after** a **GROUP BY** to compute running totals / cumulative sums over already-aggregated rows.
- A window function's own alias (e.g. an **OVER(...)**-computed average) **cannot** be referenced directly in that same query's **WHERE** clause — **WHERE** is evaluated before window functions are computed. To filter on a windowed value, wrap the query in a subquery or CTE and filter in the outer query.

Exploratory Data Analysis (EDA)

- **Purpose:** systematically analyze raw data to summarize its main characteristics, find patterns, discover anomalies, and test hypotheses — using statistical graphics and visualization.
- **Four-question framework:** (1) What is the data about? (2) What are its distributions and statistics? (3) Are there outliers or anomalies? (4) What relationships exist between variables?
- **Typical steps:** understand shape/types/metadata → check missing values/outliers → analyze distributions (numerical & categorical) → identify correlations/relationships → detect skewness (left/right/normal).
- **Visualization purposes** (the specific plotting code/syntax for these is covered later in the course):
 - **Histogram** — distribution/binning of a numerical feature.
 - **Boxplot** — detect outliers, visualize spread/variance.
 - **Bar chart** — categorical counts/trends.
 - **Scatter plot** — explore correlation between two numerical variables.
 - **Heatmap** — visualize a correlation matrix across multiple numerical features.
- **Correlation ≠ causation** — a positive or negative correlation shows variables move together, but does not by itself establish that one causes the other. Always validate data correctness and clarify unusual/unknown fields with the data owner before drawing conclusions.
- EDA is a precursor to model training — practical use cases include age-distribution analysis for marketing targeting, salary-range analysis for credit offers, and fraud detection by spotting anomalous transaction amounts/timings.

Key terms: PARTITION BY, RANK vs DENSE_RANK vs ROW_NUMBER, running total, EDA, four-question framework, correlation vs causation.

Coverage checklist

This guide covers every subtopic taught across all 14 lectures for this module — from Python fundamentals (variables, data types, conditionals, functions, scope, data structures, file/JSON handling) through the data-analysis toolkit (NumPy, pandas cleaning/aggregation/pivoting) to SQL (fundamentals, joins/subqueries/CTEs, window functions) and EDA. A few topics that were only briefly previewed or explicitly deferred to later sessions are noted inline where relevant (advanced Git usage, numeric/label encoding of categorical variables, and the specific plotting code for EDA visualizations) — you are not expected to be tested on those, but they are flagged here so you know what's coming next.